

Napredni modeli i baze podataka

Predavanja
listopad 2025.

Vektorske baze podataka



Vektorske baze podataka - teme

- 1. Zašto vektori i vektorske baze podataka**
- 2. Karakteristike vektorskih baza podataka**
- 3. Pretraga po sličnosti**
- 4. Primjena**

Zašto vektori i vektorske baze podataka

Ako korisnik želi npr. pronaći

- **članke** koji su tematski slični određenom tekstu,
- **slike** koje izgledom nalikuju nekoj slici
- **zvuk** (pjesma) nalik drugom zvuku
- **video** nalik drugom videu
- **proizvode** koji su semantički povezani s onim što je opisano u tekstualnom upitu

umjesto uspoređivanja korištenjem jednakosti primjerenija je pretraga po sličnosti značenja.

- Ideja je:
 1. podatke (tekst, slika, video...) predstaviti pomoću **vektora realnih brojeva (embeddings)**
 2. sličnost među objektima mjeriti matematičkim udaljenostima (npr. *kosinusna sličnost*, *L2* ili *Euklidska udaljenost*, *skalarni produkt*).

Vektori

- u kontekstu strojnog učenja i znanosti o podacima, vektori su **numeričke reprezentacije** podatkovnih točaka u višedimenzionalnom prostoru.
- Na primjer, riječ u obradi prirodnog jezika može biti predstavljena kao vektor od 300 realnih brojeva, čime se bilježi njezino **semantičko značenje** u tom višedimenzionalnom prostoru.
- Slično tome, slika, nakon obrade modelom dubokog učenja, može biti pretvorena u vektor koji kodira njezine značajke.
- Vektor je neproziran (opaque) objekt – niz brojeva bez izravnog značenja. Znamo da u sebi nosi značenje nekog podatka, ali ne znamo što točno znači svaki broj.
- Za vektore postoje matematičke operacije za izračun sličnosti, klasteriranje i redukciju dimenzionalnosti
- Nestrukturirani podaci predstavljeni pomoću vektora postaju pogodni za razne algoritme.

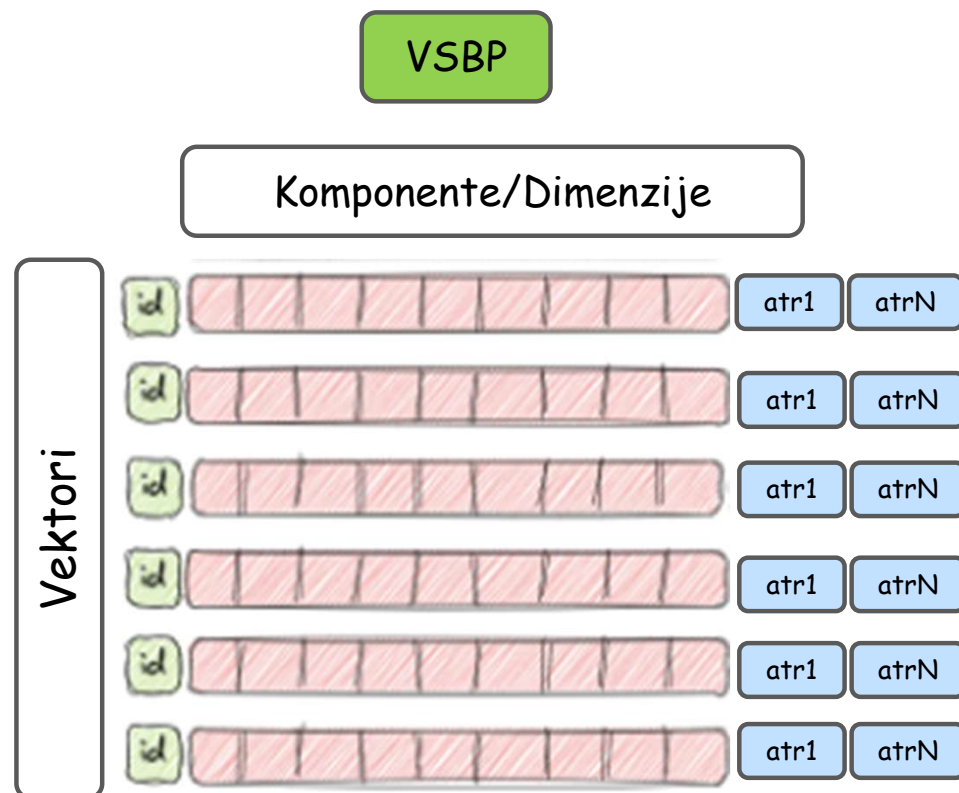
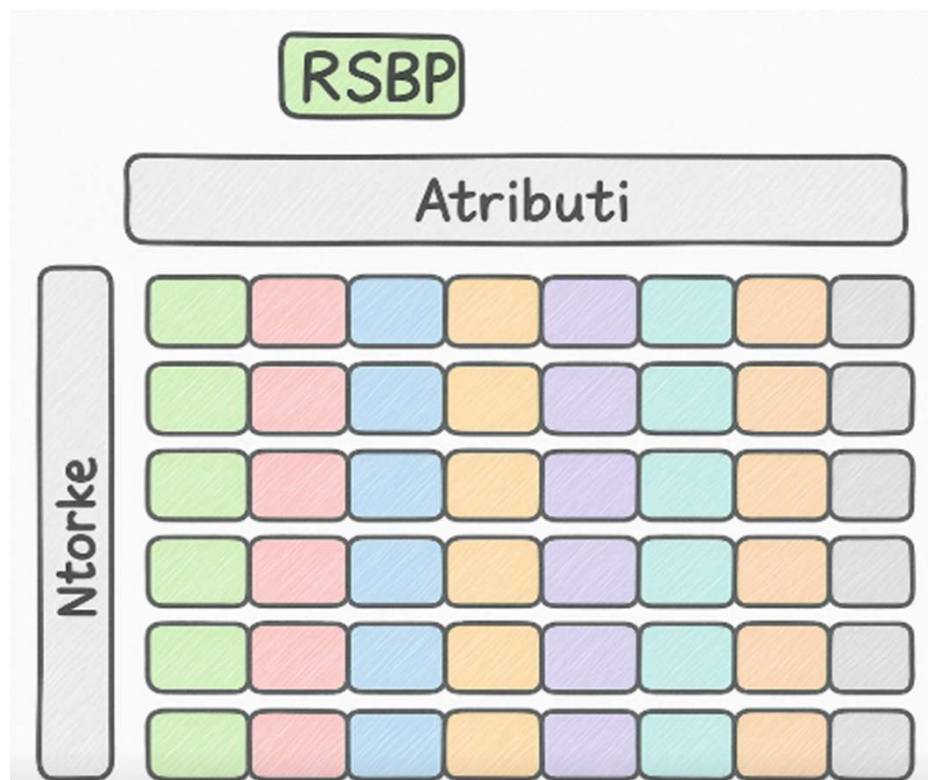
Vektorska baza podataka

- Pohrana, indeksiranje i pretraživanje informacija predstavljenih pomoću **vektora**.
- Temeljna funkcionalnost je **pretraživanje po sličnosti** (*similarity search*).
- Fokusirane na **nestrukturirane podatke** (tekst, slika, video, zvuk, ...) koji su prezentirani pomoću vektorskih ugradnji (embeddings).

Karakteristike vektorskih baza podataka

Prednosti i zahtjevi	Ograničenja i nedostaci tradicionalnih RSBP
Velike baze podataka ($N = 1M - 1000^*$ milijardi)	
Vektori s puno komponenti (stotine ili tisuće)	Mnogo međusobno povezanih relacija.
Brzo pretraživanje najbližih susjeda (milisekunde)	Ne postoji "pretraga sudsjedstva", samo komplicirani SQL upiti.
Aproksimacija (manja preciznost za bolje performanse)	Točni odgovori
Eventualna konzistentnost (manja konzistentnost je prihvatljiva)	Jaka konzistentnost, ACID svojstva transakcije.

Zapisi u relacijskoj bazi – zapisi u vektorskoj bazi



Ntorka - vektor

Ntorka / RSBP

Mali skup atributa - nekoliko desetaka.

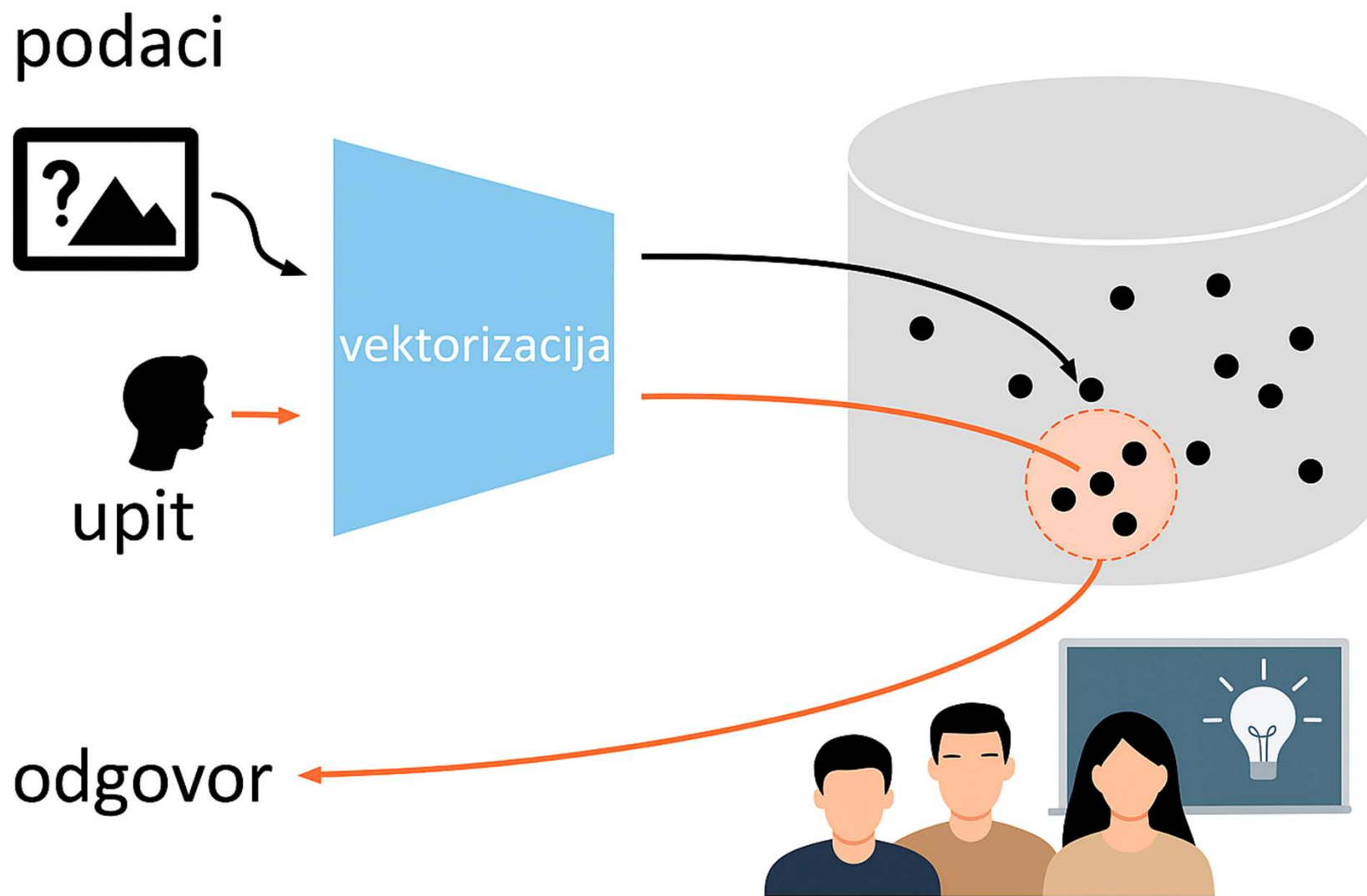
- Atributi su smisleni
 - Imaju imena.
 - Raznoliki tipovi: numerički, tekst, vremenski,...
 - Većina atributa ima prirodan poredak (npr. brojčani, abecedni).
 - Ugrađeni operatori.
 - Može se uspoređivati, selektirati, agregirati,...

Vektor / VSBP

Velika lista skalara (+ pridruženi atributi/metapodaci) - stotine do tisuće (i više) dimenzija.

- Pojedinačna komponenta (skalar) je besmislena. Značenje dolazi iz cijelog vektora.
 - Komponente nemaju imena imaju samo poziciju.
 - Uvijek realni brojevi s pomičnim zarezom.
 - Nema prirodnog poretka za vektore.
 - Uspoređivanje susjedstva i udaljenosti.

Kako funkcionira vektorska baza podataka



Kako funkcionira Vektorska baza podataka

- Prije dodavanja, podatke (npr. tekst, slika ili zvuk) je potrebno konvertirati u vektor (*embedding vector*).
- Pretvorba se najčešće radi pomoću modela strojnog učenja (poput neuronskih mreža).
- Upit je također potrebno predstaviti pomoću vektora.
- Prilikom izvršavanja upita, sustav uspoređuje vektor upita s drugim pohranjenim vektorima, identificirajući i rangirajući **najsličnije**.
- Primjenjuju se napredni algoritmi poput traženja najbližeg susjeda (ENN - Exact Nearest Neighbor) ili približno najbližeg susjeda (ANN - Approximate Nearest Neighbor).

Što je pretraživanje po sličnosti?

A	    ⇒	[1, 0, 3, 5]
B	    ⇒	[9, 4, 6, 9]
C	    ⇒	[4, 6, 2, 5]
D	    ⇒	[6, 8, 6, 1]
E	    ⇒	[7, 0, 3, 5]
F	    ⇒	[1, 4, 6, 9]
G	    ⇒	[4, 6, 2, 4]

"Pronađi dva najsličnija teksta, slike, zvuka,..."

Upit: [1, 3, 6, 8]

$$L2(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Vektor (A)	Vektor (B)	Vektor (C)	Vektor (D)	Vektor (E)	Vektor (F)	Vektor (G)
[1, 0, 3, 5]	[9, 4, 6, 9]	[4, 6, 2, 5]	[6, 8, 6, 1]	[7, 0, 3, 5]	[1, 4, 6, 9]	[4, 6, 2, 4]
[1, 3, 6, 8]	[1, 3, 6, 8]	[1, 3, 6, 8]	[1, 3, 6, 8]	[1, 3, 6, 8]	[1, 3, 6, 8]	[1, 3, 6, 8]
(L2)	(L2)	(L2)	L2	(L2)	(L2)	(L2)
$\sqrt{0^2 + (-3)^2 + (-3)^2 + (-3)^2}$					$\sqrt{0^2 + 1^2 + 3^2 + 1^2}$	
≈5.20	≈8.12	≈6.40	≈10.34	≈7.55	≈1.41	≈7.14

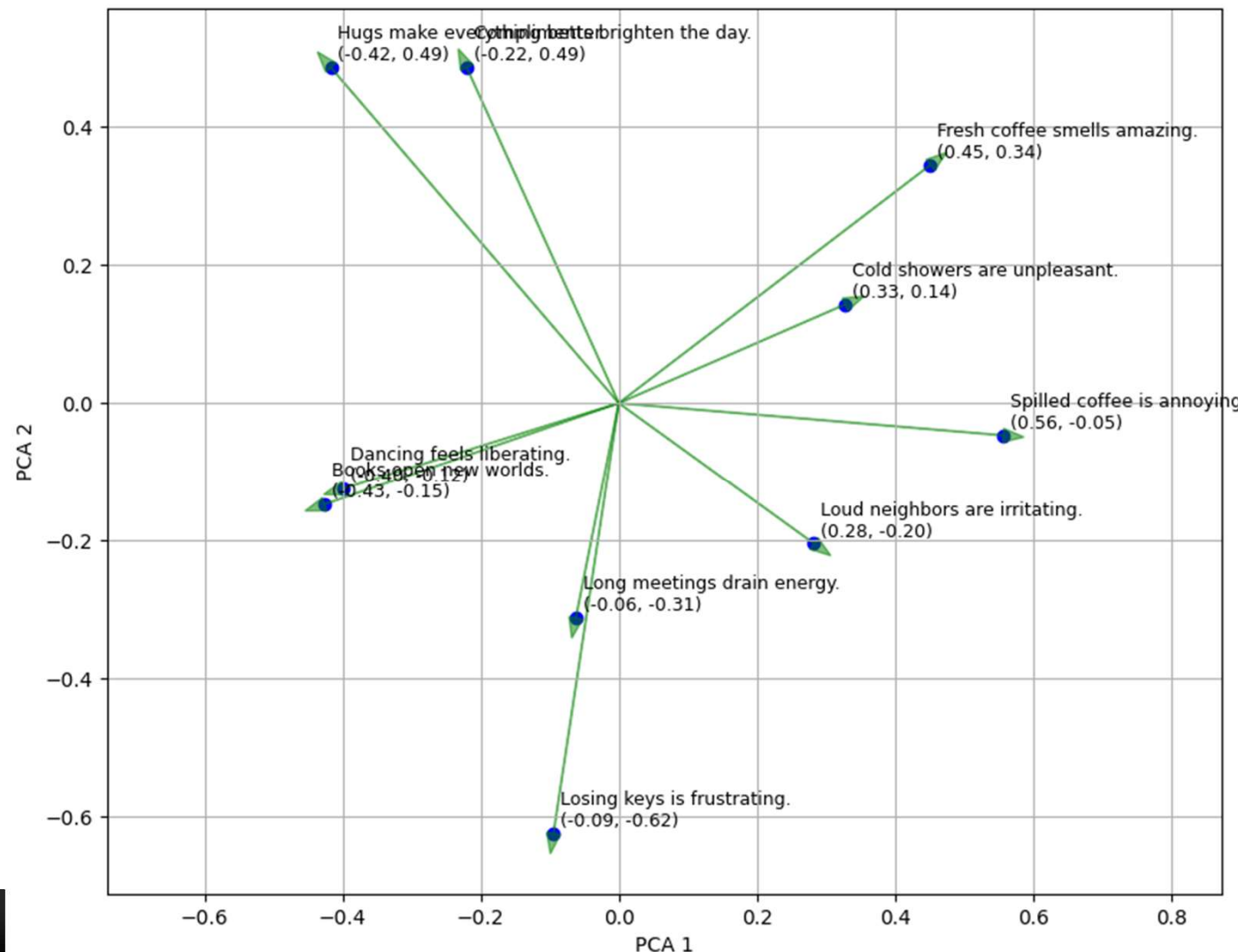
Vektorski prikaz rečenica u 2D prostoru

sentences = [

"Fresh coffee smells amazing.",
"Hugs make everything better.",
"Dancing feels liberating.",
"Books open new worlds.",
"Compliments brighten the day.",

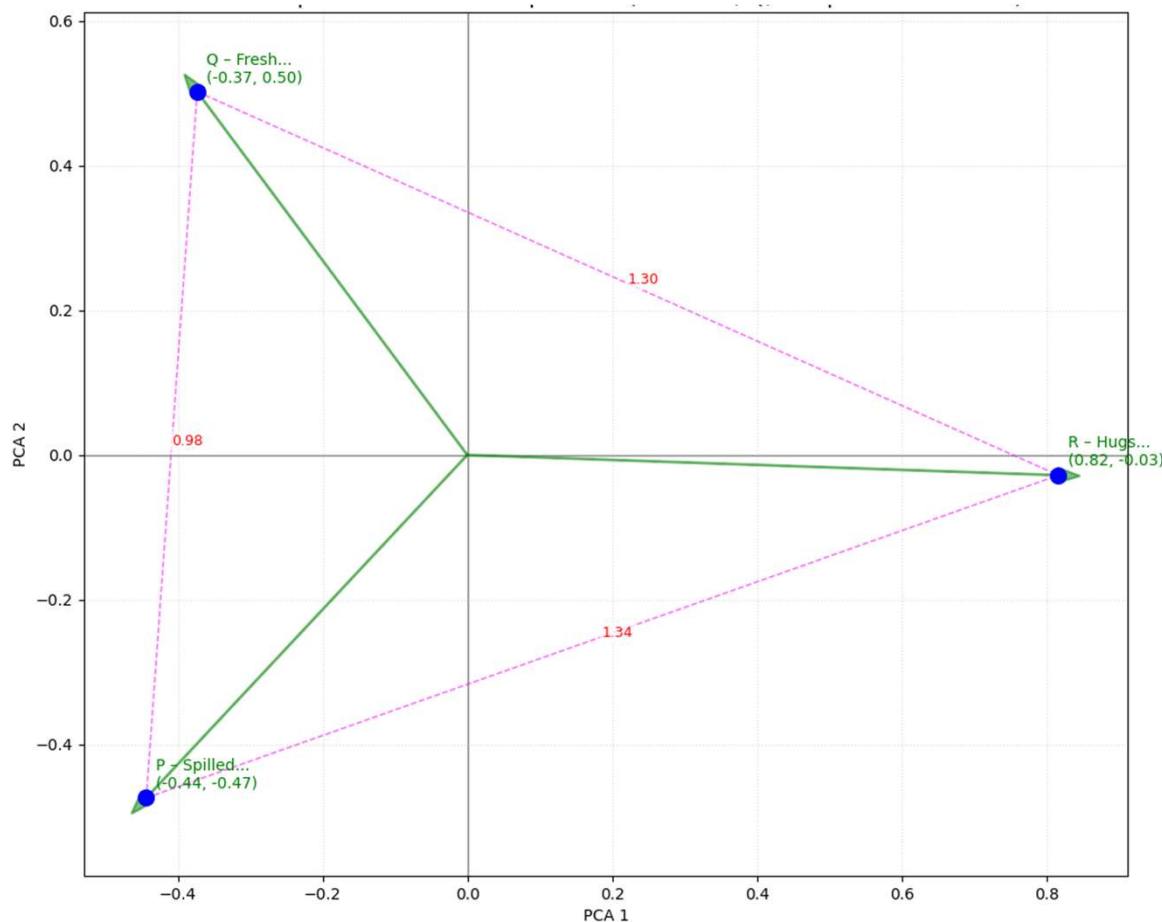
"Loud neighbors are irritating.",
"Long meetings drain energy.",
"Spilled coffee is annoying.",
"Losing keys is frustrating.",
"Cold showers are unpleasant."

- Pretvori rečenice u vektore
 - Npr. all-MiniLM-L6-v2 proizvede vektore sa 384 komponente.
- Smanji dimenzionalnost vektora da bi se rečenica mogla prikazati u 2D prostoru
 - Npr. pomoću PCA (*Principal Component Analysis*)
 - **PC1** → smjer u kojem su podaci najviše raspršeni (najveća varijanca).
 - **PC2** → smjer okomit na prvu komponentu s najvećom preostalom varijancom



L2 - Euklidska udaljenost

- Mjeri koliko su točke udaljene.
- Što je udaljenost manja, to su rečenice sličnije po značenju.



$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

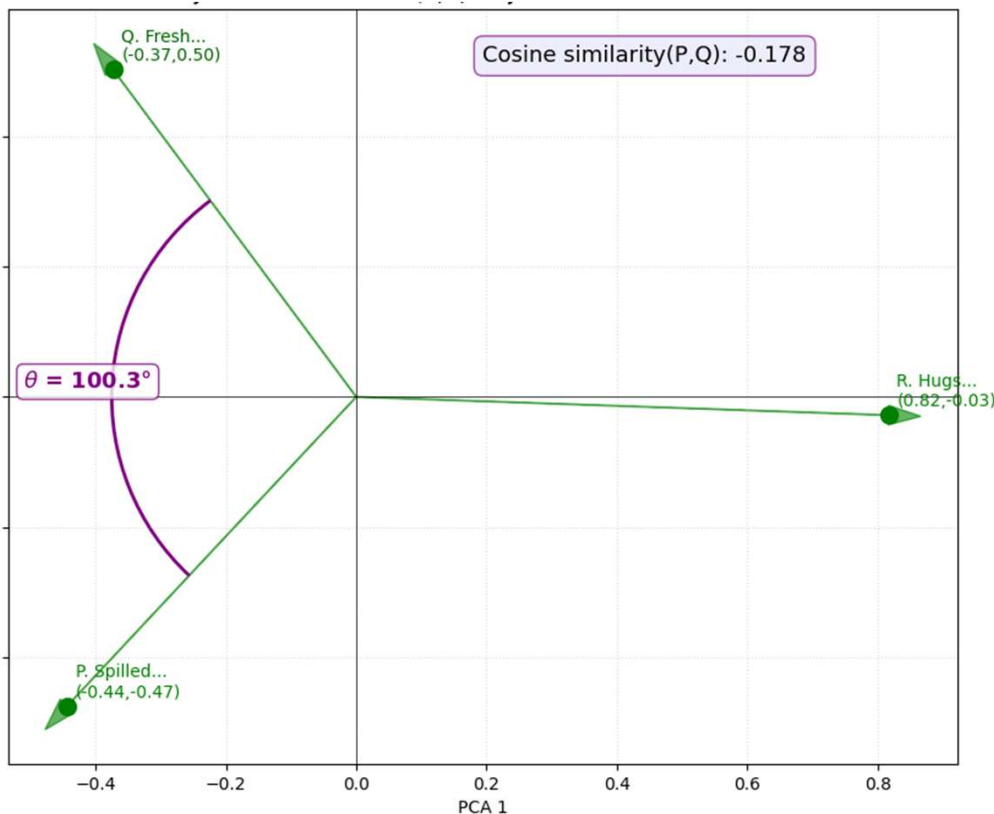
p = Spilled coffee is annoying.
q = Fresh coffee smells amazing.
r = Hugs make everything better.

$$d(p, q) = \sqrt{(-0,37 - (-0,44))^2 + (0,5 - (-0,47))^2}$$

$$d(p, q) = 0,97252$$

Kosinusna sličnost

- Mjeri koliko su dva vektora usmjerena u istom smjeru, neovisno o njihovoj duljini.
- Ako je rezultat
 - **blizu 1**, vektori su istog smjera
 - **nula**, vektori su okomiti → nema sličnosti
 - **blizu -1**, vektori su suprotno usmjereni → značenja suprotna



$$\cos(\theta) = \frac{P \cdot Q}{\|P\| \cdot \|Q\|} = \frac{\sum_{i=1}^n P_i \cdot Q_i}{\sqrt{\sum_{i=1}^n P_i^2} \sqrt{\sum_{i=1}^n Q_i^2}}$$

p = Spilled coffee is annoying.

q = Fresh coffee smells amazing.

r = Hugs make everything better.

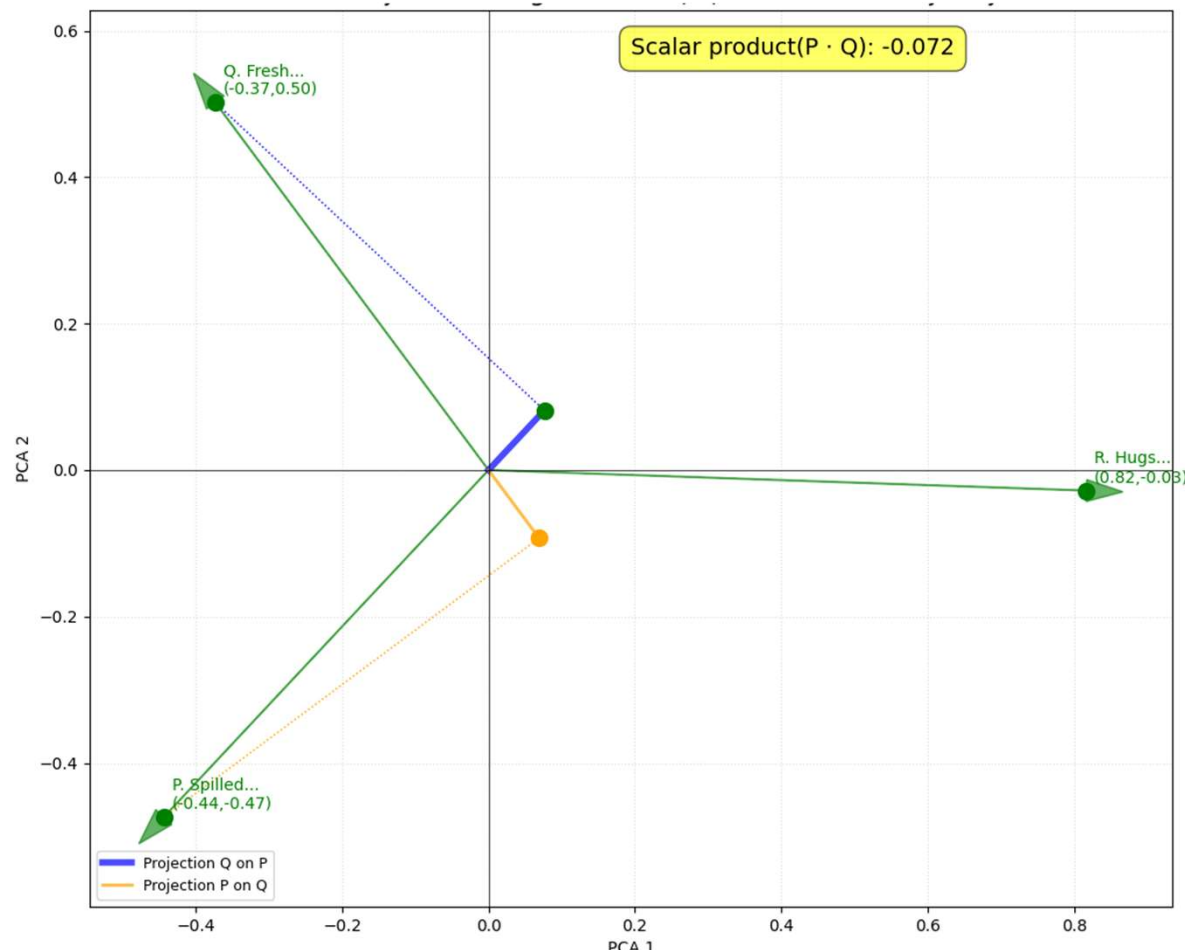
$$\cos(\theta) = \frac{(-0.44) \cdot (-0.37) + ((-0.47) \cdot 0.5)}{((-0.44)^2 + (-0.37)^2) \cdot ((-0.47)^2 + (0.5)^2)} =$$

$$\cos(\theta) = -0.178$$

$$\theta = 100.253^\circ$$

Skalarni produkt (*inner product*)

- Mjeri koliko su dva vektora usmjerena u istom smjeru, ali uzima u obzir i njihove duljine.
- Ako je rezultat
 - **velik i pozitivan**, vektori su sličnog smjera → semantički slični sadržaji
 - **nula**, vektori su okomiti → nema sličnosti
 - **negativan**, vektori su suprotno usmjereni → značenja suprotna



$$p^*q = \sum_{i=1}^n p_i * q_i$$

p = Spilled coffee is annoying.

q = Fresh coffee smells amazing.

r = Hugs make everything better.

$$p * q = -0.44 * (-0,37) + (-0,47) * 0,5$$

$$p * q = 0,0722$$

L2 – skalarni produkt – kosinusna sličnost

- **L2** (Euklidska udaljenost)
se koristi kad želimo izmjeriti koliko su vektori “udaljeni” u prostoru.
Primjer: kod mjerenja stvarne fizičke sličnosti (npr. udaljenosti između točaka ili od prosječne pogreške).
- **Kosinusna sličnost**
se koristi kad nam je važan smjer, tj. orijentacija vektora, a ne njegova duljina.
Primjer: kod usporedbe značenja rečenica (semantička sličnost), gdje duljina vektora nije bitna.
- **Skalarni** (Inner) produkt
kombinira **i veličinu i orijentaciju**.

Dvije vrste VSBP

Izvorni (Native)

- Dizajnirani posebno kao VSBP
 - Specijalizirana arhitektura
- **Primjeri:** Pinecone, Milvus/Manu, Weaviate, Azure AI Search
- Visoke performanse
- Ograničeni upiti i podatkovni model
 - Jednostavniji upiti, bez kursora, bez optimizatora, ograničen podatkovni i pohrambeni model
- Gubi se veza s izvornim podatkom

Proširenja (Extension)

- Proširuju postojeće relacijske ili NoSQL baze podataka
 - Dodatak (plugin) ili nova ugrađena funkcionalnost
- **Primjeri:** pgvector, AnalyticsDB-V, PASE, Vespa, Redis, Timescale
- Potencijalno slabije performanse
- Moćan upitni jezik i podatkovni model
 - Potpuni SQL, ACID svojstva, transakcije
- Moguće pohraniti izvor zajedno s vektorom

PostgreSQL: proširenje pgvector

1. Novi tip podatka *vector*

Broj komponenti/dimenzija ograničen na 16000.

Za primjena poput RAG-a ili semantičkog pretraživanja preporuka je koristiti vektore s onoliko dimenzija koliko ih vraćaju modeli za vektorizaciju sadržaja.

Na primjer: 384 za all-MiniLM-L6-v2, 1536 za text-embedding-ada-002 (OpenAI) dimenzije ili 1024 za e5-large-v2)

2. *Operatori*

$\langle - \rangle$	L2 distance
$\langle \# \rangle$	negative inner product (pomnožiti s -1 za inner product)
$\langle = \rangle$	Cosine distance (sličnost = 1- udaljenost)

3. *Indeksi*

HNSW

IVFFlat

HNSW ima bolje performase (omjer brzine i točnosti), ali sporije se izgrađuje i zauzima više memorije.

Može se stvoriti i bez podataka u tablici, budući da nema faze treniranja kao kod IVFFlat indeksa.

PostgreSQL: Primjer

```
CREATE EXTENSION vector;
CREATE TABLE items(
    id serial PRIMARY KEY,
    embedding vector(3));

INSERT INTO items (embedding) VALUES ('[1,2,3]');
INSERT INTO items (embedding) VALUES ('[1,3,4]');
INSERT INTO items (embedding) VALUES ('[5,2,1]');
INSERT INTO items (embedding) VALUES ('[-5,-2,-1]');
```

<->	Koliko su vektori udaljeni u prostoru
<#>	Kombinira i duljinu i smjer
<=>	Koliko su vektori <i>različiti po smjeru, bez obzira na duljinu</i>

```
SELECT *
    , items1.embedding <->
      items2.embedding l2
    , -1* (items1.embedding <#>
          items2.embedding) ij
    , 1- (items1.embedding <=>
          items2.embedding) cs
FROM items items1
CROSS JOIN items items2
WHERE items1.id <= items2.id
```

id	embedding	id-2	embedding-2	l2	ij	cs
5	[1,2,3]	5	[1,2,3]	0.000	14.000	1.000
5	[1,2,3]	6	[1,3,4]	1.414	19.000	0.996
5	[1,2,3]	7	[5,2,1]	4.472	12.000	0.586
5	[1,2,3]	8	[-5,-2,-1]	8.246	-12.000	-0.586
6	[1,3,4]	6	[1,3,4]	0.000	26.000	1.000
6	[1,3,4]	7	[5,2,1]	5.099	15.000	0.537
6	[1,3,4]	8	[-5,-2,-1]	9.274	-15.000	-0.537
7	[5,2,1]	7	[5,2,1]	0.000	30.000	1.000
7	[5,2,1]	8	[-5,-2,-1]	10.954	-30.000	-1.000
8	[-5,-2,-1]	8	[-5,-2,-1]	0.000	30.000	1.000

Indeksiranje vektora u PostgreSQL-u

1. Točno pretraživanje najbližeg susjeda (NN) (Bez indeksa)

Bez kreiranog indeksa, pgvector provodi **Točno pretraživanje najbližeg susjeda (NN)**.

- Rezultat su matematički točni najbliži susjedi - garantira **100% točnost**.
- **Nedostatak:** Može biti sporo, posebno na velikim skupovima podataka.

2. Približno pretraživanje najbližeg susjeda (ANN) (S indeksom)

Nakon kreiranja indeksa (poput IVFFlat ili HNSW), baza podataka provodi **Približno pretraživanje najbližeg susjeda (ANN)**.

- ANN žrtvuje dio točnosti rezultata za **znatno veću brzinu upita**.
- **Posljedica:** Rezultati **ne moraju biti jednaki** onima dobivenim bez indeksa (tj. ne garantira se da su *točni* najbliži susjedi).

3. Zahtjevi za indeksiranje

- Mora se kreirati **zaseban indeks** za svaku funkciju udaljenosti koju planiramo koristiti (**L2, Inner Product, Cosine Distance**).

4. Kreiranje IVFFlat indeksa

- IVFFlat indeks ima smisla kreirati tek **nakon što tablica sadrži značajnu količinu podataka**.
- Razlog tome je što IVFFlat radi klasteriranje postojećih podataka i potrebni su mu podaci za učinkovito provođenje tog početnog klasteriranja.

Naredbe za kreiranje indeksa

-- L2 udaljenost

```
CREATE INDEX ON items USING hnsw      (embedding vector_l2_ops);  
CREATE INDEX ON items USING ivfflat (embedding vector_l2_ops)  
      WITH (lists = n); --n=rows/1000 ili n = sqrt(rows)
```

-- inner product

```
CREATE INDEX ON items USING hnsw      (embedding vector_ip_ops);  
CREATE INDEX ON items USING ivfflat (embedding vector_ip_ops) WITH  
(lists = n);
```

-- kosinusna udaljenost/sličnost

```
CREATE INDEX ON items USING hnsw      (embedding vector_cosine_ops);  
CREATE INDEX ON items USING ivfflat (embedding vector_cosine_ops)  
      WITH (lists = n);
```

Primjene vektorskih baza podataka



RAG sustavi

Proširuju znanje velikih jezičnih modela (LLM) ugradnjom vanjskih izvora podataka u LLM-ove.



Pretraživanje teksta

Traženje semantički sličnih tekstova u ogromnim količinama dokumenata napisanih prirodnim jezikom.



Pretraživanje slika

Identificiranje i pretraživanje vizualno sličnih slika u velikim kolekcijama slika.



Pretraživanje videa

Pronalaženje sličnih videozapisa, scena ili objekata u velikim kolekcijama video zapisa.



Pretraživanje zvuka

Pronalaženje sličnih zvukova u velikim skupovima podataka, klasifikacija žanra ili prepoznavanja govora.



Sustavi preporuka

Usporedba ponašanja korisnika ili značajki sadržaja s drugim sličnim kako bi se kreirale učinkovite preporuke.



Detekcija anomalija

Otkrivanje podataka, događaja i opažanja koja značajno odstupaju od uobičajenog obrasca.



Multimodalno pretraživanje

Pretraživanje više vrsta podataka istovremeno, npr. teksta i slike.

RAG – poboljšanje generativnih modela

- RAG (Retrieval-Augmented Generation) je sustav koji spaja tehnike dohvaćanja informacija s generativnim modelima (poput GPT-a).
- Koristi vanjsko znanje (pohranjeno u vektorskoj bazi podataka) kako bi odgovori bili točniji i svjesniji konteksta.
- Generativni modeli (npr. GPT, LLaMA, Claude...) uče na ogromnim količinama tekstova, ali njihovo znanje je “zamrznuto” u trenutku treniranja. To znači da:
 - ne znaju ništa o novim događajima ili dokumentima nastalim nakon treniranja,
 - nemaju pristup specifičnim internim ili privatnim podacima,
 - često “izmišljaju” (haluciniraju) informacije kad ne znaju točan odgovor.

Kako funkcionira RAG

- Pohrana dokumenata:
Znanje (npr. videozapisi, slike ili dokumenti) se pretvara u vektor/ugradnje (***embeddings***) i pohranjuje u vektorsku bazu podataka.
- Dohvat (***Retreival***):
Tijekom izvršavanja korisničkog upita, identificiraju se relevantni vektori i dohvaćaju podaci.
- Proširenje (***Augmentation***):
Dohvaćene informacije prosljeđuju se velikom jezičnom modelu (LLM) kao kontekst.
- Generiranje (***Generation***):
LLM generira odgovor koristeći proširene informacije.

Uloga vektorske baze podataka u RAG sustavu

- **Vektorska baza podataka** ključna je komponenta RAG sustava jer:
- **Pohranjuje vektorske ugradnje (embeddings):** Pretvara dokumente, videozapise ili slike u **vektorske prikaze** pomoću AI modela.
- **Provodi pretraživanje po sličnosti:** Pronalazi ugradnje koje su **najbliže korisničkom upitu**, koristeći algoritme poput **k-najbližih susjeda (k-NN)** ili **približnih najbližih susjeda (ANN)**.
- Bez vektorske baze podataka, RAG **ne može učinkovito dohvatiti relevantan kontekst**.